

Chapter 8 Study Notes

Relational Algebra & Relational Calculus

Fundamentals of Database Systems (Elmasri & Navathe, 7th Ed.)

Complete Exam Preparation Guide

Contents: Core Concepts | All Operations with Examples | Exam Q&A

1. Relational Algebra — Overview

Relational Algebra is a formal query language used with relational databases. It provides the theoretical foundation for SQL and all relational query processors.

- **Procedural language:** It is a
 - You specify HOW to retrieve data step-by-step using operations.
- Operations take one or two relations as input and always return a new relation.
- This property (returning a relation) makes the algebra CLOSED — any output can be further operated on.
- A sequence of operations forms a Relational Algebra Expression.

Key Exam Point: Relational algebra is PROCEDURAL (says HOW), while Relational Calculus is DECLARATIVE (says WHAT). SQL is based on tuple calculus but uses both ideas.

1.1 Groups of Operations

Group	Operations	Notation
Unary	SELECT	σ (sigma)
Unary	PROJECT	π (pi)
Unary	RENAME	ρ (rho)
Set Theory	UNION	$R \cup S$
Set Theory	INTERSECTION	$R \cap S$
Set Theory	DIFFERENCE	$R - S$
Set Theory	CARTESIAN PRODUCT	$R \times S$
Binary	JOIN (Theta, Equi, Natural)	$R \bowtie S$
Binary	DIVISION	$R \div S$
Additional	OUTER JOIN	$R \Join S, R \ltimes S, R \Join_{\text{left}} S$
Additional	AGGREGATE FUNCTIONS	$\mathcal{F}$ SUM, COUNT, AVG, MIN, MAX

The COMPLETE SET is: $\sigma, \pi, \rho, \cup, -, \times$ — all other operations ($\cap$, JOIN, DIVISION) can be expressed using these five.

2. Unary Relational Operations

2.1 SELECT (σ)

Selects a subset of ROWS (tuples) from a relation that satisfy a condition. Works like a horizontal filter.

General Form:

$\sigma \text{ <condition> (R)}$

SELECT Expression	Meaning
$\sigma \text{ DNO}=4 \text{ (EMPLOYEE)}$	Employees in department 4
$\sigma \text{ SALARY} > 30000 \text{ (EMPLOYEE)}$	Employees earning more than 30,000
$\sigma \text{ (DNO}=4 \text{ AND SALARY}>25000) \text{ (EMPLOYEE)}$	Dept 4 employees earning > 25,000
$\sigma \text{ (DNO}=4 \text{ OR DNO}=5) \text{ (EMPLOYEE)}$	Employees in dept 4 OR dept 5

SELECT Properties (Exam-Critical)

- Result has the SAME SCHEMA (attributes) as input relation R.
- SELECT is COMMUTATIVE: $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$
- Cascade can be collapsed: $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c1 \text{ AND } c2}(R)$
- Result size: $|\text{result}| \leq |R|$ (equal only if all tuples satisfy condition)

Exam Trick: When asked to optimize, push SELECT operations down the query tree (closer to leaves) to reduce intermediate result sizes. This is called 'pushing down selections'.

2.2 PROJECT (π)

Keeps only specific COLUMNS (attributes) from a relation, discarding the rest. Works like a vertical filter. Removes duplicate tuples in the result.

General Form:

$\pi \text{ <attribute list> (R)}$

PROJECT Expression	Meaning
$\pi \text{ LNAME, FNAME, SALARY (EMPLOYEE)}$	Last name, first name, salary of all employees
$\pi \text{ SSN (EMPLOYEE)}$	SSNs of all employees
$\pi \text{ SEX, SALARY (EMPLOYEE)}$	Sex and salary columns (duplicates removed)
$\pi \text{ DNO (EMPLOYEE)}$	Department numbers (unique list)

PROJECT Properties

- Result has only the listed attributes.
- DUPLICATE TUPLES ARE REMOVED (set semantics).
- PROJECT is NOT COMMUTATIVE in general:
 - $\pi\langle\text{list1}\rangle(\pi\langle\text{list2}\rangle(R)) = \pi\langle\text{list1}\rangle(R)$ ONLY IF $\langle\text{list2}\rangle$ contains all attributes in $\langle\text{list1}\rangle$
- If the attribute list includes a KEY of R, no duplicates are eliminated, so $|\text{result}| = |R|$.
- Otherwise, $|\text{result}| \leq |R|$.

2.3 RENAME (ρ)

Renames a relation and/or its attributes. Essential when the same relation appears multiple times in a query.

Forms:

```

ρ S(B1, B2, ..., Bn) (R)    -- rename relation to S AND attributes to B1..Bn
ρ S(R)                     -- rename relation only
ρ (B1, B2, ..., Bn) (R)    -- rename attributes only

```

Example — Rename EMPLOYEE attributes to shorter names:

```

RESULT(F,M,L,S,B,A,SX,SAL,SU,DNO) <- ρ (F,M,L,S,B,A,SX,SAL,SU,DNO) (EMPLOYEE)

```

Exam use: RENAME is required when joining a table with itself (self-join), e.g., finding employees and their supervisors both from EMPLOYEE.

3. Set Theory Operations

These require TYPE COMPATIBILITY (Union Compatible): both relations must have the same number of attributes and corresponding attributes must have compatible domains.

Result attribute names are taken from the FIRST operand relation by convention.

3.1 UNION ($\cup$)

Returns all tuples that appear in R OR in S (or both). Duplicates are eliminated.

$RESULT = R \cup S$

Example — SSNs of all dept-5 employees OR supervisors of dept-5 employees:

```
DEP5_EMPS      <- σ DNO=5 (EMPLOYEE)
RESULT1        <- π SSN (DEP5_EMPS)
RESULT2(SSN)   <- π SUPER_SSN (DEP5_EMPS)
RESULT         <- RESULT1 ∪ RESULT2
```

- Commutative: $R \cup S = S \cup R$
- Associative: $R \cup (S \cup T) = (R \cup S) \cup T$

3.2 INTERSECTION ($\cap$)

Returns only tuples that appear in BOTH R AND S.

$RESULT = R \cap S$

- Commutative: $R \cap S = S \cap R$
- Can be expressed as: $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$

3.3 DIFFERENCE / MINUS ($-$)

Returns tuples in R that are NOT in S.

$RESULT = R - S$

- NOT commutative: $R - S \neq S - R$ (critical exam point!)

Example — Employees with NO dependents:

```
ALL_EMPS          <- π SSN (EMPLOYEE)
EMPS_WITH_DEPS(SSN) <- π ESSN (DEPENDENT)
EMPS_WITHOUT_DEPS <- ALL_EMPS - EMPS_WITH_DEPS
```

```
RESULT          <-  $\pi$  LNAME, FNAME (EMPS_WITHOUT_DEPS * EMPLOYEE)
```

3.4 CARTESIAN PRODUCT ($\times$)

Combines EVERY tuple of R with EVERY tuple of S. Result has all attributes of both.

$Q = R(A1..An) \times S(B1..Bm) \Rightarrow Q$ has degree $n+m$, size $nR * nS$ tuples

By itself, CROSS PRODUCT is rarely meaningful. It becomes meaningful when followed by a SELECT:

```
FEMALE_EMPS    <-  $\sigma$  SEX='F' (EMPLOYEE)
EMPNAMES        <-  $\pi$  FNAME, LNAME, SSN (FEMALE_EMPS)
EMP_DEPS        <- EMPNAMES  $\times$  DEPENDENT
ACTUAL_DEPS     <-  $\sigma$  SSN=ESSN (EMP_DEPS)
RESULT          <-  $\pi$  FNAME, LNAME, DEPENDENT_NAME (ACTUAL_DEPS)
```

This pattern (CROSS PRODUCT followed by SELECT on a join condition) is equivalent to the JOIN operation.

4. Binary Relational Operations

4.1 JOIN ($\bowtie$)

JOIN combines related tuples from two relations based on a join condition. It is shorthand for CROSS PRODUCT followed by SELECT.

$$R \bowtie \langle \text{condition} \rangle S = \sigma \langle \text{condition} \rangle (R \times S)$$

Types of JOIN

Type	Description
Theta Join (θ -join)	Any comparison operator ($=, <, >, \leq, \geq, \neq$)
Equijoin	Only equality ($=$) comparisons
Natural Join ($*$)	Equijoin on same-named attributes; removes duplicate join column

Example — Theta / Equijoin

Get each department's name and manager's info:

```
DEPT_MGR <- DEPARTMENT  $\bowtie$  MGR_SSN=SSN EMPLOYEE
```

The result DEPT_MGR contains all attributes from both DEPARTMENT and EMPLOYEE, one row per department, linked by MGR_SSN = SSN.

Example — Natural Join

Combine DEPARTMENT with DEPT_LOCATIONS on the common DNUMBER attribute:

```
DEPT_LOCS <- DEPARTMENT * DEPT_LOCATIONS
```

Implicit condition: DEPARTMENT.DNUMBER = DEPT_LOCATIONS.DNUMBER. Only ONE copy of DNUMBER appears in result.

JOIN Properties

- Result degree = $n + m$ (sum of both relations' attributes).
- Result size: 0 to $nR * nS$ tuples (usually much less).
- JOIN is not directly commutative (attribute order changes), but logically equivalent queries can be formed.

Exam shortcut: Natural join is the most common form tested. Remember — it removes the duplicate join attribute automatically.

4.2 DIVISION ($\div$)

Used to answer 'for ALL' type queries — e.g., 'Find employees who work on ALL projects of department 5.'

$$T(Y) = R(Z) \div S(X) \quad \text{where } Y = Z - X \quad (Y \text{ is } R\text{'s attributes not in } S)$$

A tuple t appears in result T if for EVERY tuple in S , there exists a corresponding tuple in R .

Example — Find employees who work on ALL projects managed by employee with SSN 123456789:

```
SMITH_PNOS      <- π PNO (σ ESSN='123456789' (WORKS_ON))
SSN_PNOS        <- π ESSN, PNO (WORKS_ON)
RESULT_ESSNS    <- SSN_PNOS ÷ SMITH_PNOS
```

Division is useful for 'universal quantification' queries. If you see 'all', 'every', or 'each' in a query, think DIVISION.

5. Additional Relational Operations

5.1 Aggregate Functions ($\mathcal{F}$)

These cannot be expressed in basic relational algebra. They compute summary statistics on groups of tuples.

Expression	Result
$\mathcal{F}$ MAX Salary (EMPLOYEE)	Maximum salary value
$\mathcal{F}$ MIN Salary (EMPLOYEE)	Minimum salary value
$\mathcal{F}$ SUM Salary (EMPLOYEE)	Total payroll sum
$\mathcal{F}$ COUNT SSN (EMPLOYEE)	Number of employees
$\mathcal{F}$ AVERAGE Salary (EMPLOYEE)	Mean employee salary
DNO $\mathcal{F}$ COUNT SSN, AVERAGE Salary (EMPLOYEE)	Count and avg salary PER department

Grouping syntax: grouping attribute is placed to the LEFT of $\mathcal{F}$:

DNO $\mathcal{F}$ COUNT SSN, AVERAGE Salary (EMPLOYEE)

Output (per department 5, 4, 1):

DNO=5: 4 employees, avg salary 33250
DNO=4: 3 employees, avg salary 31000
DNO=1: 1 employee, avg salary 55000

5.2 OUTER JOINS

Standard JOIN drops tuples with no match. OUTER JOINS preserve unmatched tuples by padding with NULLs.

Operation	Behavior
LEFT OUTER JOIN (R $\bowtie$ S)	All tuples of R kept; S attributes are NULL if no match
RIGHT OUTER JOIN (R $\bowtie$ S)	All tuples of S kept; R attributes are NULL if no match
FULL OUTER JOIN (R $\bowtie$ S)	All tuples of both R and S kept; NULL padding on either side

Example — Employees with their department names (even if department is unknown):

RESULT <- EMPLOYEE $\bowtie$ DNO=DNUMBER DEPARTMENT

Employees with no matching department get NULL in the Dname column.

5.3 Recursive Closure

Cannot be expressed in basic relational algebra. Needed for hierarchical queries like 'find all supervisees of James Borg at all levels.'

A two-level example (supervisor + their supervisees) uses:

```
SUPERVISION(Ssn1, Ssn2) <-  $\pi$  SSN, SUPER_SSN (EMPLOYEE)
RESULT1(SSN) <-  $\pi$  SSN1 ( $\sigma$  SSN2='888665555' (SUPERVISION))    -- direct
RESULT2(SSN) <-  $\pi$  SSN1 (SUPERVISION  $\bowtie$  SSN2=SSN RESULT1)    -- indirect
RESULT <- RESULT1  $\cup$  RESULT2
```

SQL3 introduced WITH RECURSIVE for general recursive closure. Basic relational algebra cannot handle arbitrary-depth recursion.

6. Worked Query Examples (COMPANY Database)

Reference schema: EMPLOYEE(Fname,Minit,Lname,Ssn,Bdate,Address,Sex,Salary,Super_ssn,Dno), DEPARTMENT(Dname,Dnumber,Mgr_ssn,Mgr_start_date), DEPT_LOCATIONS(Dnumber,Dlocation), PROJECT(Pname,Pnumber,Plocation,Dnum), WORKS_ON(Essn,Pno,Hours), DEPENDENT(Essn,Dependent_name,Sex,Bdate,Relationship)

Q1: Names and addresses of all Research dept employees

```
RESEARCH_DEPT  <- σ DNAME='Research' (DEPARTMENT)
RESEARCH_EMPS  <- RESEARCH_DEPT ⋈ DNUMBER=DNO EMPLOYEE
RESULT         <- π FNAME, LNAME, ADDRESS (RESEARCH_EMPS)
```

Single expression:

```
π Fname, Lname, Address (σ Dname='Research' (DEPARTMENT ⋈ Dnumber=Dno
EMPLOYEE))
```

Q2: Project numbers, dept number, manager's last name, birth date, address — for projects in Stafford

```
STAFFORD_PROJS <- σ Plocation='Stafford' (PROJECT)
PROJ_DEPT      <- STAFFORD_PROJS ⋈ Dnum=Dnumber DEPARTMENT
PROJ_DEPT_MGR  <- PROJ_DEPT ⋈ Mgr_ssn=Ssn EMPLOYEE
RESULT         <- π Pnumber, Dnum, Lname, Bdate, Address (PROJ_DEPT_MGR)
```

Q3: Employees with a dependent with the same first name and sex as themselves

```
EMP_DEP        <- EMPLOYEE ⋈ Ssn=Essn DEPENDENT
RESULT         <- π Fname, Lname (σ Fname=Dependent_name AND Sex=Sex (EMP_DEP))
```

Q4: SSNs of employees who directly supervise dept-5 employees

```
DEP5_EMPS      <- σ Dno=5 (EMPLOYEE)
RESULT(SSN)    <- π Super_ssn (DEP5_EMPS)
```

Q5: SSNs of employees who work in dept 5 OR supervise a dept-5 employee

```
DEP5_EMPS      <- σ Dno=5 (EMPLOYEE)
RESULT1        <- π Ssn (DEP5_EMPS)
RESULT2(SSN)   <- π Super_ssn (DEP5_EMPS)
RESULT         <- RESULT1 ∪ RESULT2
```

Q6: Employees with NO dependents

```
ALL_EMPS       <- π Ssn (EMPLOYEE)
```

```

EMPS_WITH_DEPS(SSN) <-  $\pi$  Essn (DEPENDENT)
EMPS_WITHOUT_DEPS <- ALL_EMPS - EMPS_WITH_DEPS
RESULT <-  $\pi$  Lname, Fname (EMPS_WITHOUT_DEPS * EMPLOYEE)

```

Q7: Managers with at least one dependent

```

MGR_SSNS(SSN) <-  $\pi$  Mgr_ssn (DEPARTMENT)
EMPS_WITH_DEPS <-  $\pi$  Essn (DEPENDENT)
MGR_WITH_DEPS <- MGR_SSNS  $\cap$  EMPS_WITH_DEPS
RESULT <-  $\pi$  Lname, Fname (MGR_WITH_DEPS * EMPLOYEE)

```

Q8: Employees who work on ALL projects controlled by department 5

```

DEPT5_PROJS(PNO) <-  $\pi$  Pnumber ( $\sigma$  Dnum=5 (PROJECT))
EMP_PNO <-  $\pi$  Essn, Pno (WORKS_ON)
RESULT_ESSNS <- EMP_PNO  $\div$  DEPT5_PROJS
RESULT <-  $\pi$  Fname, Lname (RESULT_ESSNS * EMPLOYEE)

```

Q9: Average salary of employees by department

```

DNO  $\mathcal{F}$  AVERAGE Salary (EMPLOYEE)

```

Q10: Count of employees and average salary overall

```

 $\mathcal{F}$  COUNT Ssn, AVERAGE Salary (EMPLOYEE)

```

7. Relational Calculus

Relational Calculus is a NON-PROCEDURAL (declarative) query language. It specifies WHAT to retrieve, not HOW.

Relational Algebra	Procedural — specifies HOW to get data
Relational Calculus	Declarative — specifies WHAT data to get

Both are relationally complete — any query in one can be expressed in the other. SQL is based on tuple relational calculus.

7.1 Tuple Relational Calculus (TRC)

Variables range over TUPLES (rows) of relations.

General Form:

```
{ t | COND(t) }
```

= 'the set of all tuples t such that COND(t) is true'

Example — Employees earning more than \$50,000:

```
{ t.Fname, t.Lname | EMPLOYEE(t) AND t.Salary > 50000 }
```

Example — Names and addresses of Research dept employees:

```
{ t.Fname, t.Lname, t.Address |  
  EMPLOYEE(t) AND (∃ d) (DEPARTMENT(d) AND  
    d.Dname='Research' AND d.Dnumber=t.Dno) }
```

Quantifiers

Quantifier	
∃ (Existential — 'there exists')	(∃ t)(F) is TRUE if F is true for at LEAST ONE tuple t
∀ (Universal — 'for all')	(∀ t)(F) is TRUE if F is true for EVERY tuple t

Example using ∀ — Employees who work on ALL projects of dept 5:

```
{ e.Lname, e.Fname | EMPLOYEE(e) AND  
  (∀ x) ( NOT PROJECT(x) OR NOT (x.Dnum=5)  
  OR (∃ w) (WORKS_ON(w) AND w.Essn=e.Ssn  
    AND x.Pnumber=w.Pno) ) }
```

Pattern for universal quantification: exclude irrelevant tuples with NOT, then assert your condition on the remainder.

7.2 Domain Relational Calculus (DRC)

Variables range over ATTRIBUTE VALUES (domains), not entire tuples. Basis for QBE.

General Form:

$\{ x_1, x_2, \dots, x_n \mid \text{COND}(x_1, x_2, \dots, x_{n+m}) \}$

Each x_i is a domain variable ranging over a single attribute domain.

Example — Birthdate and address of John B. Smith (10 variables for 10 EMPLOYEE attributes):

$\{ u, v \mid (\exists q)(\exists r)(\exists s)(\exists t)(\exists w)(\exists x)(\exists y)(\exists z)$
 $(\text{EMPLOYEE}(q, r, s, t, u, v, w, x, y, z)$
 $\text{AND } q = \text{'John'} \text{ AND } r = \text{'B'} \text{ AND } s = \text{'Smith'}) \}$

(u = Bdate, v = Address are the free variables — they appear in the result)

8. QBE — Query By Example

QBE is a graphical query language based on Domain Relational Calculus. Users fill in query templates directly in a table grid.

- P. (P dot) = 'Print this column' in the result
- Example elements begin with underscore: `_X`, `_Y` (domain variables)
- Same variable in different columns = join condition
- NOT operator before a row = negation (exclusion)

QBE Notation	Meaning
P. in a column	Include this attribute in the result
Condition >20 in a cell	Filter: value must be > 20
John in Fname cell	Filter: Fname must equal 'John'
Same <code>_X</code> in two rows	Join condition: same value in both rows
NOT mark before a row	Exclude tuples matching this row
G. in a cell (e.g. G._DX)	Group by this attribute
P.CNT.ALL in a cell	Count all rows
P.AVG.ALL in a cell	Average of all values

Example — Q1 (Research dept employees): Fill in DEPARTMENT row: Dname='Research', Dnumber=`_DX`. Fill in EMPLOYEE row: Fname=P._FN, Lname=P._LN, Address=P._Addr, Dno=`_DX`. Place results in RESULT table with P._FN, P._LN, P._Addr.

Example — Q6 (Employees with no dependents): In EMPLOYEE row put P. in Fname and Lname, `_SX` in Ssn. In DEPENDENT row put `_SX` in Essn. Place a NOT mark ($\neg$) before the DEPENDENT row.

QBE exam tip: Two rows with the SAME variable in a column express an AND join. Two rows with DIFFERENT variables in Essn column express an OR condition for that attribute.

9. Exam-Type Questions with Full Solutions

All queries use the COMPANY database. Read each question carefully before checking the solution.

Q1. [MCQ] Which operation is used to select specific columns from a relation?

A) SELECT (σ) B) PROJECT (π) C) JOIN ($\bowtie$) D) RENAME (ρ)

ANSWER: B) PROJECT (π)

Explanation: SELECT filters rows based on a condition. PROJECT filters columns. These are the most commonly confused operations in exams.

Q2. [MCQ] Which of the following is NOT a property of the SELECT operation?

- A) SELECT is commutative
- B) A cascade of SELECTs can be combined into one with AND
- C) SELECT removes duplicate tuples from the result
- D) The result schema is the same as the input relation

ANSWER: C) SELECT removes duplicate tuples from the result

Explanation: PROJECT removes duplicates, not SELECT. SELECT simply filters rows; the result may still contain duplicates (in multi-tuple databases).

Q3. [MCQ] What is required for UNION, INTERSECTION, and DIFFERENCE operations?

- A) The relations must have at least one common attribute
- B) The relations must be type compatible (union compatible)
- C) One relation must be a subset of the other
- D) Both relations must have the same number of tuples

ANSWER: B) The relations must be type compatible (union compatible)

Explanation: Union compatible means: same number of attributes AND corresponding attributes have compatible domains. The number of tuples does NOT matter.

Q4. [True/False] The DIFFERENCE operation ($R - S$) is commutative.

ANSWER: FALSE

Explanation: $R - S$ returns tuples in R not in S. $S - R$ returns tuples in S not in R. These are generally different results.

However, UNION and INTERSECTION ARE commutative ($R \cup S = S \cup R$, $R \cap S = S \cap R$).

Q5. [True/False] The result of a PROJECT operation always has the same number of tuples as the input.

ANSWER: FALSE

Explanation: PROJECT removes duplicate tuples. So $|\pi_{\langle \text{list} \rangle}(R)| \leq |R|$.

Exception: if the attribute list includes a KEY of R, then no duplicates exist, so the count equals $|R|$.

Q6. [Short Answer] What is the 'complete set' of relational algebra operations? Why is it called complete?

ANSWER: The complete set is: SELECT (σ), PROJECT (π), RENAME (ρ), UNION ($\cup$), DIFFERENCE ($-$), CARTESIAN PRODUCT ($\times$)

It is called 'complete' because any relational algebra expression can be expressed using only these six operations.

Other operations are just shorthand/derived:

INTERSECTION: $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$

JOIN: $R \bowtie_{\langle c \rangle} S = \sigma_{\langle c \rangle}(R \times S)$

Q7. [Short Answer] Explain the difference between EQUIJOIN and NATURAL JOIN.

EQUIJOIN: A join where ALL join conditions use only equality ($=$). The result includes BOTH copies of the join attribute(s).

NATURAL JOIN: A special equijoin on attributes with the SAME NAME in both relations. The result includes only ONE copy of the join attribute(s).

Example:

EQUIJOIN: $\text{DEPT} \bowtie \text{DEPT.MGR_SSN}=\text{EMP.SSN}$ EMP -- includes both MGR_SSN and SSN columns

NATURAL JOIN: $\text{DEPT} * \text{DEPT_LOCATIONS}$ -- only one DNUMBER column in result

Q8. [Query] Write a relational algebra expression to find the names (Fname, Lname) of all female employees who have at least one dependent.

Step 1: Get female employees

$\text{FEMALE_EMPS} \leftarrow \sigma_{\text{Sex}='F'}(\text{EMPLOYEE})$

Step 2: Get SSNs of employees who have dependents

$\text{EMPS_WITH_DEPS}(\text{SSN}) \leftarrow \pi_{\text{Essn}}(\text{DEPENDENT})$

Step 3: Natural join to get female employees who have dependents

$\text{FEMALE_WITH_DEPS} \leftarrow \text{FEMALE_EMPS} \bowtie_{\text{Ssn}=\text{SSN}} \text{EMPS_WITH_DEPS}$

Step 4: Project names

$\text{RESULT} \leftarrow \pi_{\text{Fname}, \text{Lname}}(\text{FEMALE_WITH_DEPS})$

Single expression:

$\pi_{\text{Fname}, \text{Lname}}(\sigma_{\text{Sex}='F'}(\text{EMPLOYEE}) \bowtie_{\text{Ssn}=\text{Essn}} \text{DEPENDENT})$

Q9. [Query] Find the name and salary of employees who earn more than their direct supervisor.

This requires a self-join — we need RENAME to use EMPLOYEE twice:

```
EMP (EN, EM, EL, ES, EB, EA, EX, ESAL, ESUP, EDN) <- ρ EMP (EMPLOYEE)
SUP (SN, SM, SL, SS, SB, SA, SX, SSAL, SSUP, SDN) <- ρ SUP (EMPLOYEE)

RESULT <- π EN, EL, ESAL (σ ESAL > SSAL (EMP ⋈ ESUP=SS SUP))
```

Meaning: Join employee (EMP) to their supervisor (SUP) on EMP.Super_ssn = SUP.Ssn,
then select only rows where employee salary > supervisor
salary,
then project the employee's first name, last name, and salary.

Q10. [Query] Using aggregate functions, find the number of employees and total salary cost for each department.

```
DNO ℱ COUNT Ssn, SUM Salary (EMPLOYEE)
```

This groups tuples by DNO and applies aggregate functions within each group.
Result: one row per department with (DNO, count of employees, sum of salaries).

To rename the output columns:

```
ρ R(Dno, Emp_Count, Total_Salary) (DNO ℱ COUNT Ssn, SUM Salary  
(EMPLOYEE))
```

Q11. [Query] Write the tuple relational calculus expression equivalent to Q8 (female employees with dependents).

```
{ t.Fname, t.Lname |  
  EMPLOYEE(t) AND t.Sex='F' AND  
  (∃ d) (DEPENDENT(d) AND d.Essn = t.Ssn) }
```

Q12. [Hard] Explain when and why you would use the DIVISION operation. Write a query to find employees who work on ALL projects located in 'Houston'.

Use DIVISION when queries contain universal quantification ('for all', 'every', 'each').
Division $R(Z) \div S(X)$ gives tuples in R that are paired with ALL tuples in S.

Step 1: Find project numbers of all Houston projects

```
HOUSTON_PROJS(Pno) <- π Pnumber (σ Plocation='Houston' (PROJECT))
```

Step 2: Get (Essn, Pno) pairs from WORKS_ON

```
EMP_PNOS <- π Essn, Pno (WORKS_ON)
```

Step 3: Divide — get Essns paired with ALL Houston project numbers

```
RESULT_ESSNS <- EMP_PNOS ÷ HOUSTON_PROJS
```

Step 4: Get names

```
RESULT <- π Fname, Lname (RESULT_ESSNS ⋈ Essn=Ssn EMPLOYEE)
```

Q13. [Query] Write a relational algebra expression to retrieve the names of all employees in department 5 who work more than 10 hours per week on project number 10.

```
DEPT5_EMPS <- σ Dno=5 (EMPLOYEE)
PROJ10_WORK <- σ Pno=10 AND Hours>10 (WORKS_ON)
JOINED <- DEPT5_EMPS ⋈ Ssn=Essn PROJ10_WORK
RESULT <- π Fname, Lname (JOINED)
```

Single expression:

```
π Fname, Lname (σ Dno=5 (EMPLOYEE) ⋈ Ssn=Essn σ Pno=10 AND Hours>10 (WORKS_ON))
```

Q14. [Conceptual] What is the difference between LEFT OUTER JOIN and INNER JOIN? When would you use each?

INNER JOIN (regular $\bowtie$): Returns only tuples that have a MATCHING tuple in the other relation. Tuples with no match are DROPPED.

LEFT OUTER JOIN ($R \bowtie\!\!\!\bowtie S$): Returns ALL tuples from the LEFT relation R. If no match exists in S, S attributes are padded with NULL.

Use INNER JOIN when: you only want related data (e.g., employees who have a department).

Use LEFT OUTER JOIN when: you want ALL records from one side regardless of matches

(e.g., all employees even if their department record is missing).

Example:

```
INNER: EMPLOYEE ⋈ Dno=Dnumber DEPARTMENT -- drops employees with no dept
```

```
LEFT: EMPLOYEE ⋈\!\!\!\bowtie Dno=Dnumber DEPARTMENT -- keeps all employees, NULL for no dept
```

Q15. [Algebra vs Calculus] Write both the relational algebra AND tuple calculus expression for: Find Fname and Lname of all employees who work on project named 'ProductX'.

RELATIONAL ALGEBRA:

```
PRODX <- σ Pname='ProductX' (PROJECT)
PRODX_WORK <- PRODX ⋈ Pnumber=Pno WORKS_ON
RESULT <- π Fname, Lname (PRODX_WORK ⋈ Essn=Ssn EMPLOYEE)
```

TUPLE RELATIONAL CALCULUS:

{ t.Fname, t.Lname |

```
EMPLOYEE(t) AND  
( $\exists$  w) (WORKS_ON(w) AND w.Essn=t.Ssn AND  
      ( $\exists$  p) (PROJECT(p) AND p.Pname='ProductX'  
            AND p.Pnumber=w.Pno)) }
```

Key difference: Algebra specifies step-by-step HOW to get data; Calculus just states WHAT conditions the result must satisfy.

10. Quick Reference Summary

Operation	Symbol	Quick Description
SELECT	$\sigma_{\langle \text{cond} \rangle}(R)$	Filter ROWS by condition
PROJECT	$\pi_{\langle \text{attrs} \rangle}(R)$	Keep specified COLUMNS; remove duplicates
RENAME	$\rho_S(B1..Bn)(R)$	Rename relation/attributes
UNION	$R \cup S$	All rows in R OR S (union-compatible required)
INTERSECTION	$R \cap S$	Rows in BOTH R AND S
DIFFERENCE	$R - S$	Rows in R NOT in S (not commutative!)
CARTESIAN PRODUCT	$R \times S$	All combinations; result size = $ R \times S $
JOIN (Theta)	$R \bowtie_{\langle \text{cond} \rangle} S$	$= \sigma_{\langle \text{cond} \rangle}(R \times S)$
NATURAL JOIN	$R * S$	Equijoin on same-named attrs; 1 copy of join attr
DIVISION	$R(Z) \div S(X)$	Rows in R paired with ALL rows in S
LEFT OUTER JOIN	$R \Join S$	All R rows; NULL for non-matching S attrs
AGGREGATE	$G \mathcal{F} \text{func}(\text{attr})(R)$	SUM/COUNT/AVG/MIN/MAX, optionally grouped by G

Key Distinctions to Memorize

- SELECT vs PROJECT: SELECT = filter ROWS; PROJECT = filter COLUMNS.
- UNION vs JOIN: UNION stacks rows (same schema); JOIN combines columns (different schemas).
- EQUIJOIN vs NATURAL JOIN: Natural join removes the duplicate join attribute.
- DIFFERENCE is NOT commutative. UNION and INTERSECTION ARE.
- PROJECT removes duplicates; SELECT does NOT.
- Algebra = Procedural (HOW). Calculus = Declarative (WHAT). SQL = Calculus-based.
- DIVISION answers 'for all' queries. Think: 'employees who work on EVERY project in dept 5.'
- The complete set is: σ , π , ρ , $\cup$, $-$, $\times$ (only 6 operations; everything else is derived).

Final Exam Advice: Always write queries in step-by-step form first (intermediate relations), then optionally convert to a single expression. This reduces errors and earns partial credit.